Flight Ticket Booking Website

Detailed Requirement Plan with Definitions and Explanations

Adithyan

September 30, 2026

Contents

1 Goal and Scope **3**

1.1 Two Points of View 3

2 User Roles **4**

3 Functional Requirements **4**

3.1 Traveller Features (MVP) 4

3.2 Traveller Features (Phase 2) 5

3.3 Agent Portal Features 6

3.4 Admin Panel Features 6

4 Non-Functional Requirements **7**

5 Suggested Technology Stack **8**

6 Data Model **9**

7 Key APIs **10**

8 Booking Flow **11**

9 Third-Party Integrations **11**

10 Development Phases **12**

11 Team Roles **12**

12 Risks and Solutions **13**

13 Success Metrics **14**

14 Recommended MVP **14**

1 Goal and Scope

The goal of this project is to build a website where travellers can search flights between cities, compare airlines and fares, choose seats and extras, pay online, and receive an e-ticket by email and SMS. At the same time, travel agents get a tool to book on behalf of their customers, and the platform owner gets control over pricing, payments, refunds, and reports. The scope is divided into connected parts.

Traveller website The customer-facing website (also usable on mobile browsers) where people search flights, book tickets, and manage trips. It is the most visible part of the product, so its speed, clarity, and trustworthiness decide whether users come back. For example, a traveller should be able to go from the home page to a paid, confirmed ticket in about three minutes.

Agent portal A web dashboard for travel agents and small agencies who book for many customers. They get a wallet, quick booking tools, and commission reports. Without it, the platform would depend only on direct customers.

Admin panel A web dashboard used only by the platform owner and staff. It controls the whole business: managing suppliers, setting markups and service fees, handling refunds and complaints, and viewing reports. Think of it as the control room.

Backend The server-side software that holds the business rules and data. When a traveller taps "Search", the browser does not know any flights; it asks the backend, which queries airline and distribution systems and sends back the combined answer. The backend also enforces rules such as "re-check the price before payment" and "issue the ticket only after payment is confirmed".

API (Application Programming Interface) A defined set of requests and responses through which the website and the backend talk to each other, and through which the backend talks to airline systems. For example, the site sends "flights from Chennai to Delhi on 12 November, one adult", and the API replies with a structured list of flight options.

Flight supplier The external source of flight inventory and fares: an airline's own system or a distribution network. The platform does not own aircraft or seats; it sells seats that suppliers make available. This is the biggest difference from a bus or movie booking system.

1.1 Two Points of View

The same system looks different to the people who use it and the people who build it. This plan describes both.

	Traveller point of view	Developer point of view
Main question	Can I find a good fare and book it safely?	How do we connect suppliers, keep prices correct, and run it reliably?
Cares about	Fast search, clear prices, easy changes and refunds	Supplier APIs, fare validation, data, security, uptime
Covered in	Sections 2, 3, 8	Sections 4 to 13

2 User Roles

A role defines who a person is in the system and what they are allowed to see and do. Separating roles keeps the system safe and easy to use, because each person sees only the tools that matter to them.

Traveller The end customer who searches, books, pays, changes, cancels, and checks in. A traveller can see only their own bookings and saved details. Travellers are the main source of revenue, so their experience must be the smoothest.

Travel agent A professional who books for many customers. Agents keep a prepaid wallet, see net fares and commission, and can book and manage many tickets from one dashboard.

Support agent A staff member who helps customers through calls or chat. Agents can look up bookings, help with changes and cancellations, and record complaints. Their access is limited so that they cannot change pricing or financial settings.

Admin The owner or management team. Admins configure suppliers and markups, approve agents, manage refunds, and read reports. Because this role is powerful, it must be protected with strong passwords and two-step verification.

3 Functional Requirements

Functional requirements describe what the system must do. Each feature below is something a user can directly see or use.

3.1 Traveller Features (MVP)

The MVP (Minimum Viable Product) features are the essentials without which the site cannot work as a booking platform.

Sign up and login Users can book as a guest with an email and phone number, or create an account using an OTP (one-time password) sent by SMS or email. Accounts let users see past bookings and save traveller details. The OTP should expire after a few minutes.

Flight search The core action of the site. The user enters origin, destination, departure date, return date (for round trips), number of adults, children, and infants, and cabin class. Helpful touches include airport auto-suggestions while typing (city names and airport codes), a swap button, and a flexible-date calendar showing the lowest fare per day.

Trip types One-way, round trip, and (later) multi-city trips. A round trip may show outbound and return flights together so that the user can pair them.

Search results A list of flight options showing airline, flight number, departure and arrival times, duration, number of stops, and price. Results should appear progressively, because different suppliers answer at different speeds.

Filters and sorting Filters narrow the list by stops (non-stop, one stop), airline, departure time window, price range, and duration. Sorting orders it by cheapest, fastest, or earliest. Together they help users decide quickly when there are dozens of options.

Flight details and fare rules Expanding a result shows the route with layovers, aircraft type, baggage allowance, and the fare rules: whether the fare is refundable, the change and cancellation fees, and meal or seat inclusions. Clear rules prevent disputes later.

Fare types Airlines sell the same flight in several fare types, such as Saver, Flexi, and Business. The site shows the differences side by side so that the user knows exactly what each price includes.

Traveller details For each passenger the site collects title, first name, last name, date of birth, and gender, plus contact email and phone. Names must match the traveller's government ID exactly, because airlines may refuse boarding on a mismatch. International trips also need passport number, expiry, and nationality.

Seats, baggage, and meals (add-ons) Optional extras chosen after passenger details: seat selection from a seat map, extra baggage, and meals. Each add-on shows its price, and the total updates immediately.

Price review A summary before payment with the base fare, taxes, fees, add-ons, coupon discount, and the final amount. If the price has changed since the search, the user is told clearly before continuing.

Payment The user pays with UPI, debit or credit card, net banking, or a wallet. The total must be visible before paying. A reliable payment flow is critical: if it fails or confuses the user, the booking is lost.

E-ticket and itinerary After successful booking, the site shows a confirmation with the PNR (Passenger Name Record, the airline's booking reference), passenger names, flight details, and baggage. The e-ticket is also sent by email and SMS and can be downloaded as a PDF.

Notifications Automatic messages: booking confirmation, payment failure, schedule change or cancellation by the airline, web check-in reminder, and departure reminder.

My Bookings A page listing upcoming, completed, and cancelled trips. From here the user can view the ticket, download the invoice, request a change, or cancel.

Cancellation and refund Lets the user cancel a ticket and shows the refund amount, based on the fare rules and the airline's cancellation fee, before confirming. It also shows when the money will reach the account. Transparent rules reduce complaints.

3.2 Traveller Features (Phase 2)

These features improve the product and help it grow, but they are added after the core booking flow is stable.

Date change (rebooking) Lets the user move a ticket to another date or flight. The system checks availability, and shows the fare difference and change fee before the user agrees.

Web check-in Guides the user through airline online check-in and provides the boarding pass link or file, saving time at the airport.

Price alerts and fare calendar The user asks to be told when the fare for a route drops below a chosen amount. It brings users back to the site.

Fare hold Reserves a seat and price for a short period, for a small fee, so the user can decide without losing the fare.

Flight status Shows live status: on time, delayed, gate, and terminal, using a flight data provider.

Saved travellers and profile Stores frequent co-travellers and passport details securely so the next booking takes seconds.

Coupons and offers Promotional codes for discounts or cashback, with rules for validity, minimum amount, and usage limits.

Loyalty points and wallet Points or stored balance earned from bookings and refunds, usable for later purchases.

Multi-city and multi-language support Multi-city trips, and a site available in several languages, for example English, Hindi, Tamil, and Telugu.

Support chat Chat or call linked to a specific booking so agents see the context at once.

Corporate booking Company accounts with approval flows and consolidated invoices.

3.3 Agent Portal Features

Agent registration and approval Agents submit business documents, which are verified before they can sell tickets. This protects travellers and the platform's reputation.

Wallet A prepaid balance that the agent tops up and that is deducted for every ticket. It speeds up booking and reduces payment failures.

Quick booking Search and book on behalf of a customer using net fares, with the option to add the agent's own service fee.

Commission and reports Statements showing bookings, commission earned, cancellations, and wallet transactions.

3.4 Admin Panel Features

Supplier management Configure which suppliers are active, their credentials, and their priority. A supplier can be switched off if it has problems.

Markup and service fee The extra amount added on top of the supplier fare, as a percentage or fixed fee, possibly different per airline, route, or user type. This is usually the main source of income of a booking platform.

Booking management View and search every booking, retry failed ticketing, and handle special cases by hand.

Refunds and payouts Approve and track refunds, and record every money movement for accounting.

Banners, coupons, and content Create promotional banners, discount codes, and help content without needing a developer.

Dispute management Track complaints such as wrong charges, cancelled flights, and refund delays until they are resolved. A history of each case helps fair decisions.

Reports Dashboards of bookings, revenue, top routes, conversion, cancellations, and supplier performance.

Users, roles, and audit log Manage staff accounts and permissions, and keep a permanent record of important actions.

4 Non-Functional Requirements

Non-functional requirements describe how well the system works rather than what it does. Users may not notice them when they are done right, but they immediately notice when they are missing.

Fare validation (re-pricing) Flight prices and seat availability change every few minutes. Before payment, the system must ask the supplier again for the exact price and availability of the chosen flight. If the price changed, the user must be told and must agree before paying. This is the most important rule of the system, because without it the platform could sell tickets at a loss or promise seats that no longer exist.

Performance How quickly the site responds. The first search results should appear within about three seconds, and the full list within about ten. Techniques such as caching popular routes, parallel supplier calls, and streaming results help maintain speed.

Timeouts and graceful degradation Supplier systems can be slow or down. The site must give up waiting after a set time, show results from the suppliers that answered, and never freeze because one supplier failed.

Scalability The ability to keep working as users grow. Traffic rises sharply during holidays, festivals, and airline sales. A scalable design can add more servers automatically for such peaks.

Security Protecting user data, payments, and the system from unauthorised access, with secure login, encryption, protection against common attacks, and role-based permissions.

HTTPS The secure version of the web protocol. It encrypts everything between the browser and the server, so passwords, OTPs, and passport details cannot be read by someone spying on the network.

PCI compliance PCI DSS is the international standard for handling card data. The safest approach is to let the payment gateway collect and store card details so that the platform never touches them.

Protection of personal data Passport numbers, dates of birth, and contact details are sensitive. They must be encrypted, shown only to those who need them, and removed according to a retention policy.

Reliability The system must end in a consistent state even under failure. If the internet drops during payment, either the ticket is issued and paid, or the payment is refunded, never something in between.

Availability (uptime) The percentage of time the service works. A target of 99.9 percent allows only about nine hours of downtime in a year, achieved with backups, monitoring, and multiple servers.

Time zones and currency Flight times are always shown in the local time of each airport, with correct day changes on overnight flights. Prices are shown in the user's currency, with clear rounding rules.

Accessibility and compatibility Usable with keyboard and screen readers (WCAG 2.1 AA), and working on the latest two versions of major browsers and on all screen sizes.

5 Suggested Technology Stack

A technology stack is the combination of tools and languages used to build the product. The choices below are widely used, well documented, and have large developer communities, which makes it easier to find help.

Layer	Options	Explanation
Website	React or Next.js	Builds the traveller site and the agent and admin dashboards. Next.js adds features that help pages load fast and appear well in search engines, which matters for route pages.
Styling	CSS, Tailwind CSS	Consistent, responsive design on all screens.
Backend	Node.js, Django, or Spring Boot	Frameworks for the server logic. Node.js is fast to start with, Django has many built-in features, and Spring Boot is common in large enterprise systems.
Database	PostgreSQL	Stores data in linked tables and supports transactions, meaning a group of changes is saved completely or not at all. Essential for bookings and money.
Cache	Redis	An in-memory store used for cached searches, sessions, and short-lived price quotes with automatic expiry.
Search data	PostgreSQL or OpenSearch	Fast airport and airline lookup for auto-suggestions.
Message queue	RabbitMQ or Kafka	Handles background jobs such as ticket issuance retries, e-ticket emails, and reconciliation without slowing the user.
Flight suppliers	Amadeus, Sabre, Travelport, or airline APIs	Distribution systems and airline interfaces that provide flights, fares, booking, and ticketing.
Payments	Razorpay, Stripe, or PayU	Gateways that process UPI, cards, and net banking and notify the backend of the result.

Layer	Options	Explanation
Messaging	MSG91, Twilio, SendGrid, or Amazon SES	Send OTP, SMS, and email tickets.
File storage	Amazon S3 or similar	Stores e-ticket PDFs and invoices safely and cheaply.
Hosting and DevOps	AWS, GCP, Docker, CI/CD	Rented cloud servers, packaging of the app, and automated testing and deployment.
Monitoring	Grafana, Prometheus, or Sentry	Shows system health and alerts the team when errors or slow supplier calls appear.

6 Data Model

The data model describes what information the database stores and how the pieces relate. A good model makes the system easier to build and prevents errors such as duplicate or missing data.

Users Account details: name, email, phone, and verification status.

Travellers Saved co-travellers with name, date of birth, gender, and optional passport details, linked to a user.

Airports and airlines Reference data: airport codes, city, country, time zone, airline codes, and names. Used for search suggestions and display.

Search sessions and fare quotes A short-lived record of what was searched and the exact fare offered, with an expiry time. It is the proof of what price the user was shown.

Bookings A reservation with booking ID, status (pending, confirmed, ticketed, cancelled, failed), contact details, total amount, and time of creation.

Flight segments Each flight leg of a booking, with airline, flight number, origin, destination, departure and arrival times, and cabin class. A trip with a stop has more than one segment.

Passengers The travellers on a booking with name, age type (adult, child, infant), gender, and travel document details.

Tickets The PNR and the ticket number for each passenger, issued after payment.

Add-ons Selected seats, baggage, and meals with prices.

Payments Each payment attempt with amount, method, gateway reference, and result. Failed attempts are stored too, for investigation and reconciliation.

Refunds Money returned to the user, linked to a cancellation, with airline fee, service fee, refund amount, and status.

Suppliers and markups Supplier settings and the markup and fee rules applied to fares.

Agents and wallets Agent accounts, wallet balances, and every wallet transaction.

Coupons, alerts, and reviews Discount codes with rules, price alerts set by users, and feedback about trips.

Notifications and audit logs A log of messages sent and of important staff actions.

7 Key APIs

An API endpoint is a specific address on the backend that performs one task. Below are the most important ones, each explained with its purpose.

Send OTP and Verify OTP The first sends a code to the user's phone or email. The second checks it and returns a login token.

Refresh token A login token is short-lived for safety. This endpoint issues a new one in the background so the user stays logged in.

Airport suggestions Returns matching airports and cities as the user types. It is called very often, so it must be very fast and cached.

Search flights Accepts origin, destination, dates, passengers, and cabin, calls the suppliers in parallel, and returns combined and sorted results. It is the most used and most expensive endpoint, so caching, timeouts, and streaming are essential.

Get fare rules Returns baggage, change, and cancellation rules for the selected fare.

Reprice and validate Asks the supplier for the exact current price and availability of the selected flight before payment, and returns a new quote if anything changed.

Create booking Saves the booking with a pending status, with passenger details, and holds the seat with the supplier where possible.

Payment webhook A webhook is an automatic message sent by the payment gateway to the backend, reporting whether a payment succeeded or failed. The backend must trust this message, after verifying its signature, rather than only the browser, because the user might close the page right after paying.

Issue ticket Internal call to the supplier to issue the ticket after payment succeeds, returning the PNR and ticket numbers. If it fails, it is retried automatically.

Get booking and Download ticket Returns booking details and generates the e-ticket or invoice file.

Cancel booking and Refund quote Shows the refund amount under the fare rules, then cancels with the supplier and starts the refund through the payment gateway.

Add-ons and seat map Returns the seat layout and add-on prices, and saves the user's choices.

Flight status Returns live status from a flight data provider.

Admin and agent CRUD CRUD stands for Create, Read, Update, and Delete. These endpoints let admins and agents manage suppliers, markups, coupons, wallets, and content.

8 Booking Flow

The booking flow is the step-by-step journey of a traveller from the home page to a ticket. Making this journey short and clear is the most important design goal.

1. **Search.** The user enters origin, destination, dates, passengers, and cabin, and taps Search. The system queries suppliers and shows results as they arrive.
2. **Select flight and fare.** The user compares flights by price, time, stops, and airline, opens the fare rules, and chooses a fare type.
3. **Price re-check.** The system asks the supplier for the exact current price and availability. If the price changed, the user is told and can accept or go back.
4. **Traveller details.** The user enters passenger names exactly as on their ID, contact details, and, for international trips, passport details.
5. **Add-ons.** The user optionally chooses seats, baggage, and meals, and can apply a coupon.
6. **Payment.** The user reviews the final amount and pays. The payment gateway processes the payment securely.
7. **Ticketing and confirmation.** After payment succeeds, the backend issues the ticket with the supplier, saves the PNR, and shows the confirmation. The e-ticket is sent by email, SMS, and notification.
8. **Failure cases.** If payment fails or times out, no ticket is issued and the user can try again. If payment succeeds but ticketing fails, the system retries automatically, and if it still fails, the money is refunded in full and the user is informed at once.

9 Third-Party Integrations

A third-party service is a product built by another company that you connect to your platform instead of building it yourself. This saves time and usually gives better quality than a first-time in-house version.

Flight distribution systems Global systems such as Amadeus, Sabre, and Travelport supply flights and fares from many airlines through one integration. Some airlines can also be connected directly through their own APIs, often at lower cost.

Payment gateway Handles secure collection of money and refunds through UPI, cards, net banking, and wallets.

SMS and OTP provider Delivers login codes and booking messages quickly and reliably.

Email service Services such as SendGrid or Amazon SES send tickets and invoices without them landing in spam.

Flight status provider A data service that supplies live delay, gate, and terminal information.

Currency and tax services Supply exchange rates where multiple currencies are shown, and support correct GST (Goods and Services Tax) on service fees.

Fraud check tool Scores payments for risk so that suspicious bookings can be reviewed before ticketing.

10 Development Phases

Splitting the work into phases makes the project easier to plan, track, and control.

Phase	Duration	What happens
1. Planning and design	2–3 weeks	Finalise requirements, choose suppliers, draw wireframes (simple layout sketches), create the visual design, and decide the system architecture, which is the overall structure of how the parts fit together.
2. Supplier integration and backend	5–7 weeks	Connect to flight suppliers, and build search, re-pricing, booking, payment, and ticketing logic, testing each API on its own.
3. Traveller website	4–6 weeks	Build all traveller pages and connect them to the APIs, handling loading states, slow suppliers, and errors properly.
4. Agent and admin panels	3–5 weeks	Build dashboards for suppliers, markups, bookings, refunds, wallets, and reports.
5. Testing	3–4 weeks	Functional testing, load testing for many searches at once, payment and ticketing edge cases, security testing, and accessibility checks.
6. Launch	1–2 weeks	Set up live servers, move suppliers to their production mode, publish the site, and prepare monitoring.
7. Post-launch	Ongoing	Fix bugs, study user feedback, and add Phase 2 features step by step.

Because some phases can run in parallel, for example the pages can be built while the backend is being finished, a small team can usually deliver an MVP in about four to six months. Access to real supplier systems often takes extra time for agreements and certification.

11 Team Roles

Product owner Decides what should be built and in what order, and represents the customers and the business.

UI/UX designer UI is how the site looks; UX is how it feels to use. The designer creates layouts, colours, and flows that are easy for anyone to understand.

Frontend developer Turns the designs into working pages and connects them to the backend.

Backend developer Builds the server, database, business rules, and payment and ticketing logic.

Integration engineer Specialises in connecting supplier APIs, mapping their formats into one common format, and handling their errors.

QA tester QA means Quality Assurance. The tester tries the site in many ways to find bugs and confirms that new changes do not break old features.

DevOps engineer Sets up and maintains servers, automatic deployment, backups, and monitoring so the site stays online.

12 Risks and Solutions

Every project has risks. Identifying them early makes it possible to prepare solutions before they become real problems.

Price or seat changes before payment *Problem:* the fare shown in search is no longer available when the user pays. *Solution:* always re-price with the supplier before payment, show any change clearly, and set a short expiry on every quote.

Payment success but ticketing failure *Problem:* the money is taken, but the ticket is not issued because of a supplier or network error. *Solution:* rely on payment webhooks, retry ticketing from a queue, run a reconciliation job that regularly compares payment records with booking records, and refund automatically if ticketing cannot be completed.

Slow or unavailable suppliers *Problem:* a supplier system is slow or down, delaying search or booking. *Solution:* parallel calls with timeouts, partial results, caching of popular routes, and the ability to switch a supplier off from the admin panel.

Wrong passenger names *Problem:* names differ from the ID, and the airline refuses boarding or charges a correction fee. *Solution:* clear on-screen guidance, validation of allowed characters, a confirmation screen showing names, and a clear correction policy.

Payment fraud and chargebacks *Problem:* stolen cards are used for bookings and the platform loses the money later. *Solution:* fraud scoring, OTP or 3-D Secure authentication, limits on suspicious behaviour, and manual review of high-risk bookings.

Complex refunds *Problem:* refund amounts depend on airline rules, timing, and taxes, and mistakes cause disputes. *Solution:* show the refund quote before cancelling, store the fare rules used at the time of booking, and track every refund to completion.

Bots and scraping *Problem:* bots send huge numbers of searches, which increases supplier costs and slows the site. *Solution:* rate limits, which cap the number of requests a user or device may send in a given time, plus caching and bot detection.

Legal and compliance *Problem:* selling travel needs proper legal setup. *Solution:* register the business, follow supplier and airline rules, handle GST correctly, publish terms of use, a privacy policy, and a clear refund policy, and follow data protection law.

13 Success Metrics

Metrics are numbers that show whether the product is doing well. Tracking them from day one helps in making decisions based on facts rather than guesses.

Search-to-booking conversion The percentage of searches that lead to a paid booking. A low rate signals problems in prices, choice, or the flow.

Look-to-book ratio The number of searches for every booking. Suppliers charge or limit heavy searching, so this ratio must be watched.

Payment success rate The percentage of payment attempts that succeed. If low, the payment screen or gateway needs attention.

Ticketing failure rate The percentage of paid bookings where ticket issuance fails on the first attempt. It should be as close to zero as possible.

Price-change rate at re-check How often the price changes between search and payment. A high rate frustrates users and suggests stale caching.

Search speed Time to first results and to complete results.

Cancellation and refund time The share of tickets cancelled and how long refunds take. Slow refunds damage trust.

Repeat customers The number of users who book more than once. It shows whether people trust the site enough to return.

14 Recommended MVP

An MVP (Minimum Viable Product) is the smallest working version of the product that delivers real value to users. Instead of building every feature and launching after a year, the MVP is launched early, real users try it, and their feedback decides what to build next. This lowers risk and cost.

For this website, the recommended MVP includes:

- **Domestic one-way and round-trip search** – start with popular routes and one or two suppliers.
- **Results with filters and sorting** – the heart of the product.
- **Fare details and price re-check** – protects both the user and the business.
- **Traveller details and OTP or guest booking** – fast and simple.
- **UPI or card payment** – reliable online payment with clear failure handling.
- **E-ticket by email and SMS** – an instant ticket after payment.
- **My Bookings, cancellation, and refund** – with a transparent policy.

For a learning or college project, the flight suppliers can be replaced by a mock flight API or sample data files, and the payment gateway by its test mode, so the entire flow can be built and

demonstrated without real supplier agreements or real money.

Features such as date change, web check-in, price alerts, fare hold, loyalty, multi-city, corporate booking, and multiple languages are added after launch, once the core booking flow is stable and users are booking regularly.